

The Chaotic Torus Protocol (CTP): A GPU-Oriented Diffusion Layer for Symmetric Encryption

Eric



Abstract—The Chaotic Torus Protocol (CTP) is a research prototype for a symmetric transport cipher whose diffusion layer is built around a 3-dimensional lattice update rule designed to run efficiently on massively parallel hardware. This document describes the current design: a dual-plane architecture separating an authenticated encrypt-then-MAC Data Plane from a standard AEAD Control Plane (a resilience property once thought to require active Control Plane participation was later found achievable entirely within the Data Plane, discussed below); a Feistel-structured lattice diffusion layer with a nonlinear step of verified algebraic degree; a KangarooTwelve-based keystream extraction sponge chosen specifically for its parallelizable tree-hash structure; a connection-identification layer supporting multiplexing and migration; and GPU compute-kernel implementations of both the lattice update and the underlying Keccak-p permutation, validated for correctness against independent CPU reference implementations and measured on real GPU hardware. Both components now measure faster on GPU than an optimized sequential CPU baseline: batched lattice diffusion across 128 concurrent sessions ran at approximately $88\times$ the throughput of sequential CPU execution, and batched keystream extraction, after a sequence of profiling-driven fixes described in detail below, reached $1.3\text{--}2.0\times$ the throughput of an optimized CPU implementation at 512–8192 concurrent sessions, with the margin still growing at the largest scale tested. Getting there required finding and fixing six distinct defects across several rounds of measurement, most of them invisible until the previous fix exposed what was underneath it — an early, incomplete version of the same optimization measured $230\text{--}1500\times$ slower, and profiling later showed a further “fixed” version’s flat performance ratio was actually $90\%+$ unaccelerated Python/numpy overhead rather than GPU hardware saturation, which is not the conclusion a less granular measurement would have supported. Two further defects were found in the connection-identification layer by directly attacking it rather than assuming it safe, including an off-path address-spoofing vulnerability. This document also gives CTP’s Data Plane a formal, reduction-based security argument — nonce-based authenticated encryption security following from standard encrypt-then-MAC composition, and a separate soundness argument for the connection-identification layer’s path validation — each conditional on stated PRF/MAC assumptions rather than unconditional, with the least-supported assumption further probed (not proven) by a cube-tester attack and an exact differential distribution table analysis, both described in detail below. Despite the internal GPU speedup, CTP’s absolute keystream-extraction throughput (under 1 Gbps aggregate at the largest scale tested) remains one to two orders of magnitude below a single CPU core running contemporary hardware-accelerated AES-256-GCM or ChaCha20-Poly1305, and several further orders of magnitude below dedicated hardware offload; this comparison is reported explicitly rather than left for the reader to infer. This draft states plainly what has and has not been established: correctness of every cryptographic construction described here has been verified against independent references; the

performance results are measured on one GPU configuration, not characterized broadly; the formal security argument is conditional on assumptions no third party has reviewed; and no third-party cryptanalysis has been performed. CTP should be treated as a research artifact, not a candidate for protecting real data.

Index Terms—Symmetric Encryption, GPU Computing, Cellular Automata, Authenticated Encryption, Parallel Cryptography, Connection Migration, Formal Security Proofs.

1 INTRODUCTION

Modern symmetric ciphers are designed primarily around single-core and SIMD performance. CTP explores a different question: if a cipher’s diffusion layer is built from operations that are naturally data-parallel — a large lattice of cells, each updated by a small, uniform, local rule — can that layer be run efficiently on a GPU, and does that change what a session-oriented cipher can afford in terms of security margin and concurrent throughput? This document describes the current state of that design. It does not claim CTP is competitive with, or a replacement for, standardized primitives such as AES-GCM or ChaCha20-Poly1305, which remain the appropriate choice for protecting real traffic today.

2 ARCHITECTURE OVERVIEW

CTP is an ephemeral, symmetric session protocol deployed inside a tunnel initialized via a Post-Quantum Key Encapsulation Mechanism (e.g., ML-KEM). A session splits into two planes:

- 1) **The Data Plane:** payload encryption combining a Feistel-structured lattice diffusion layer with encrypt-then-MAC authentication.
- 2) **The Control Plane:** a standard AES-GCM channel, key-ratcheted via HKDF, used for session setup. An earlier draft of this design also assigned it sequence-recovery duties; Section 7 reports that this specific resilience property turned out to be achievable entirely within the Data Plane instead, without active Control Plane participation, which is retained as a real finding rather than quietly corrected away.

Both planes are fully authenticated; no plaintext-affecting data is accepted by either plane without prior tag verification.

3 SESSION SETUP AND KEY DERIVATION

Each session derives three independent subkeys from a 256-bit master key and a 128-bit session nonce, plus a dedicated epoch key described in Section 7:

$$K_{\text{diff}} = \text{HKDF}(K_{\text{master}}, \text{"diffusion"} \parallel \text{nonce}) \quad (1)$$

$$K_{\text{auth}} = \text{HKDF}(K_{\text{master}}, \text{"auth"} \parallel \text{nonce}) \quad (2)$$

$$K_{\text{perturb}} = \text{HKDF}(K_{\text{master}}, \text{"perturb"} \parallel \text{nonce}) \quad (3)$$

The nonce is generated randomly per session and communicated once during session setup (e.g., carried in the KEM handshake transcript). Binding every derived key to both the master key and the nonce means that a master-key reuse event does not by itself collapse two sessions to the same keystream — the nonce must also repeat, which is a substantially narrower and more defensible operational requirement than depending on the master key alone.

4 THE DIFFUSION LAYER

The diffusion layer operates on a $64 \times 64 \times 64$ boolean lattice, seeded from K_{diff} and evolved by a Feistel-structured round function.

4.1 Round Function

The lattice is split into two halves along one axis, L and R . Each round computes

$$L' = R, \quad R' = L \oplus F(R), \quad (4)$$

where F combines a linear diffusion step and a nonlinear step:

$$\theta(x) = x \oplus \bigoplus_{\text{6 axis-neighbors}} \quad (5)$$

$$\chi(x) = x \oplus (a \wedge b) \oplus (c \wedge d \wedge e) \quad (6)$$

followed by XOR with a public, generation- and round-indexed constant (breaking translational symmetry across rounds, the same purpose Keccak’s *iota* step serves). Eight such rounds are applied per lattice generation.

This construction is invertible *by design*, regardless of whether F itself is a bijection: the Feistel structure guarantees a unique inverse as long as it is correctly implemented, which has been confirmed by round-trip testing at full lattice scale. This matters because χ ’s specific nonlinear form is not itself bijective at this lattice size, and no simple reparameterization makes it so — wrapping it in a Feistel network sidesteps that limitation entirely rather than working around it.

4.2 Why Three Dimensions

The round function updates each lattice cell using only that cell and a small number of fixed-offset neighbors, all read from the state at the start of the round — no cell’s update depends on another cell’s update within the same round. This is an embarrassingly parallel workload: 131,072 independent, identical computations per lattice half per round, with no branching. A 3D lattice specifically (rather than 1D or 2D) gives each cell more independent neighbors to couple to per round, which is what lets a fixed number of Feistel rounds achieve full-lattice diffusion faster than a lower-dimensional structure would.

4.3 Measured Properties

The nonlinear step’s algebraic degree is 3, confirmed by brute-force computation of its Algebraic Normal Form (Zhegalkin transform) over its 6-input local truth table, not merely asserted from the form of the expression. Density and avalanche were measured at full lattice scale: population holds at 49.7–50.2% over 100 generations with no drift, and a single-bit seed difference produces 49.9% average bit-difference in the resulting lattice state within one generation.

5 KEYSTREAM EXTRACTION

Keystream is extracted via a sponge construction absorbing the ratchet state, the packed lattice bytes, and the packet sequence number, squeezed to the required keystream length:

$$\text{Keystream}_i = \text{K12}(S_i \parallel L_i \parallel \text{seq}_i, \text{len}) \quad (7)$$

KangarooTwelve (K12) was chosen specifically for its Sakura tree-hash structure: for inputs above 8192 bytes, K12 splits the input into independent chunks, computes a 32-byte chaining value per chunk, and combines them in a final node — the per-chunk computations are fully independent and can be executed in parallel. K12 also uses a 12-round reduced Keccak-p permutation rather than the 24-round Keccak-f used in SHA-3/SHAKE, which is faster per byte even without parallelism, at a deliberately smaller (though still comfortable, per its designers’ published analysis) security margin.

The ratchet itself advances every packet regardless of how often the lattice evolves:

$$S_i = \text{SHA3-256}(S_{i-1} \parallel \text{SHA3-256}(M_{i-1}) \parallel \text{seq}_i) \quad (8)$$

which is what guarantees keystream uniqueness per packet; the lattice’s own evolution schedule (by default, once every 16 packets) is an amortization of its computational cost and does not weaken this guarantee.

6 AUTHENTICATED ENCRYPTION

Payload encryption is encrypt-then-MAC:

$$C_i = M_i \oplus \text{Keystream}_i \quad (9)$$

$$\text{Tag}_i = \text{HMAC-SHA3-256}(K_{\text{auth}}, \text{seq}_i \parallel C_i) \quad (10)$$

A receiver verifies Tag_i before decrypting and discards any packet that fails verification, without further processing. A 64-epoch sliding replay window (sequence-based) is checked only after tag verification succeeds, so forged or corrupted packets cannot be used to manipulate replay-window state.

7 RESILIENCE: EPOCH-BASED SELF-HEALING

The ratchet chain $S_i = \text{SHA3-256}(S_{i-1} \parallel \text{SHA3-256}(M_{i-1}) \parallel \text{seq}_i)$ has no gap tolerance by construction. Direct testing (dropping a single packet in an otherwise correctly-operating session) confirmed this is not a graceful-degradation edge case: every packet after a single loss decrypts to silent, undetected garbage for the rest of the session. The tag does not catch this, since it is keyed by a fixed session key independent of ratchet state and verifies correctly on every genuinely-sent

packet regardless of whether the receiver’s ratchet has desynchronized.

Every `epoch_size` packets (64 by default), both the ratchet state and the lattice reset to values derived deterministically from a dedicated epoch key K_{epoch} and the epoch number alone, not chained from any prior packet. Since the epoch number is $\lfloor \text{seq}/\text{epoch_size} \rfloor$ and `seq` is visible in the cleartext header, either endpoint independently computes the correct epoch-start state without having processed anything from the previous epoch. This bounds a single loss event’s blast radius to at most `epoch_size`−1 packets rather than the remainder of the session, and needs no request/response exchange to do it — there is no message the mechanism could itself get stuck waiting on. Testing confirmed the no-loss baseline, single-loss recovery at the next epoch boundary, and the honest boundary of what this does not fix: packet *reordering* (as opposed to loss) within one epoch, which would require abandoning within-epoch chaining, a larger change not undertaken here.

This mechanism turned out to resolve more than originally intended: the dual-plane split in Section 2 was motivated in early drafts by giving the Control Plane an active role in sequence recovery after loss. It does not need one — the epoch mechanism above achieves this entirely within the Data Plane, with the Control Plane left to session setup alone. That the stronger, simpler outcome arrived by testing the original design’s assumption rather than by planning for it from the start is itself worth recording, not editing away.

8 CONNECTION IDENTIFICATION AND PATH VALIDATION

CTP’s wire format carries no session identifier at all, only a sequence number — a receiver has no way to know which session’s keys apply to a packet except by assuming the network layer beneath it already provides that mapping. This blocks connection migration, load-balancer routing, and multiplexing multiple sessions over one channel, the same problem QUIC’s Connection ID solves by decoupling session identity from the network address.

A connection ID is derived per epoch as $\text{CID}_e = \text{trunc}_{16}(\text{SHA3-256}(K_{\text{epoch}} \parallel \text{"CTP-CID"} \parallel e))$, reusing the epoch mechanism already in place rather than adding a separate rotation scheme: since both endpoints already derive K_{epoch} independently, a receiver can precompute upcoming epochs’ CIDs and register them ahead of time, with no extra handshake messages needed for rotation. Correctness was tested directly: interleaved packets from multiple sessions route to the correct session by CID, and rotation across epoch boundaries works with no additional signaling.

A first, naive implementation updated a session’s trusted address immediately on any packet bearing a recognized CID. Rather than assume this was fine, the specific attack it predicts was tested directly: an attacker with *no key material at all* intercepts one genuine in-flight packet and relays a copy of it from a different claimed address, ahead of the real delivery. It worked exactly as predicted — the spoofed address was accepted, and the legitimate sender’s real packet, arriving moments later, was rejected as a replay. This requires

no cryptographic capability, only the ability to observe and relay packets.

The fix, analogous to QUIC’s challenge/response path validation: an address change is not trusted until it proves possession of K_{auth} by computing $\text{HMAC-SHA3-256}(K_{\text{auth}}, \text{"CTP-PATH-CHALLENGE"} \parallel R)$ for a fresh random challenge R . Packets from an unvalidated address are not decrypted at all until validation succeeds, so they cannot consume replay-window state either — closing the collateral denial-of-service along with the address hijack. Re-running the identical attack against the fix: rejected cleanly, trusted address unchanged, legitimate delivery unaffected. A separate test confirms a genuine migration (the real endpoint answering correctly) still succeeds. Review of this mechanism found one more gap: unbounded challenge issuance let an attacker claiming a third party’s address cause a fresh challenge to fire at that victim on every packet. A per-address cooldown closes this; it is a resource-management control, not a cryptographic one, and is not part of the soundness argument below.

9 FORMAL SECURITY ARGUMENT

No symmetric primitive — AES, SHA-3, or CTP’s own lattice — has ever been proven secure from unconditional first principles; confidence in each rests on public cryptanalysis failing to do better than brute force, not on a proof that no attack exists. What can be proven is a *reduction*: that if specific building blocks behave as assumed, the composed construction inherits security from them. This section builds that reduction for CTP’s Data Plane and is explicit about where the proof ends and an unproven, project-specific assumption begins.

Threat model. The adversary is network-level and computationally bounded: it sees all ciphertext and cleartext sequence numbers, and may drop, delay, reorder, duplicate, or inject packets adaptively; it does not have direct key access, and side-channel access (timing, power, GPU memory) is out of scope, matching the open item in Section 12. It is nonce-respecting by definition of the security notion below — the proof does not cover (key, nonce) reuse, which is exactly the two-time-pad failure mode addressed earlier as an operational requirement, not a proof obligation. Two things follow that are worth stating rather than leaving implicit: CTP provides *no forward secrecy* against master-key compromise, since every derived value, including every epoch’s start state, is a direct deterministic function of the master key; and the epoch-based resync mechanism is an availability property, not a confidentiality or integrity one, and is outside the scope of the proof below.

Construction. Let $G_{K_{\text{diff}}}(\text{St}_i)$ denote the keystream extraction function, where St_i is the ratchet-and-lattice state. K_{diff} and K_{auth} are independently derived from the master key via HKDF.

Assumption 1. No efficient adversary distinguishes $G_{K_{\text{diff}}}$ from a random function, given that St_i never repeats under a fixed key — which holds by construction here (a strictly monotonic sequence number, chained through a one-way hash of every prior plaintext), not as a further assumption; the earlier two-time-pad analysis directly confirmed distinct (key, nonce) pairs never collide.

Assumption 2. HMAC-SHA3-256, keyed independently of K_{diff} , is a strongly unforgeable MAC, supported by Bellare’s proof that HMAC is a secure PRF under an assumption weaker than collision resistance on its underlying hash — a far better-supported assumption than Assumption 1, since SHA-3 has received a decade of public cryptanalysis that CTP’s own lattice construction has not.

Result. Under Assumption 1, a standard hybrid argument shows $C_i = M_i \oplus G_{K_{\text{diff}}}(\text{St}_i)$ is indistinguishable from a one-time pad (replacing each G output with independent true randomness, at a cost of a factor of q — the query count — in the reduction, since no St_i repeats). Under Assumptions 1 and 2, the encrypt-then-MAC generic composition theorem (Bellare and Namprempre, 2000) — proven specifically for this composition order, which is what CTP implements, tag verified before decryption — gives CTP’s Data Plane nonce-based authenticated encryption security, with advantage bounded by $q \cdot \epsilon_{\text{PRF}}$ plus the MAC’s own forgery advantage.

9.1 Validating Assumption 1: A Cube-Tester Attempt

The proof above is only as strong as Assumption 1, which is specific to this project and unreviewed by anyone else: no third party has examined whether combining the lattice’s evolving state with K12 extraction preserves K12’s own security argument. Rather than restate the general-purpose randomness testing already reported, a cube tester (Dinur and Shamir, 2009) was built and aimed specifically at the failure mode this project has repeatedly flagged as its central risk — low algebraic degree surviving to the output — applied to the *composed* keystream generator (ratchet, lattice, and K12 extraction together), not chi in isolation, whose local degree was already established exactly by brute-force computation.

For a chosen cube dimension k , k bit positions of the master key are chosen as cube variables; the full CTP encryption pipeline runs for all 2^k assignments to those bits at a fixed base point, and the output keystream bit is XOR-summed across all 2^k runs. Sub-maximal algebraic degree in those variables would show up as a systematic bias in that sum across independent trials; a maximal-degree system shows none.

TABLE 1
Cube tester results, full CTP keystream generator

k	Trials	Result	p -value
7	100	47/100 ones	0.617
9	50	29/50 ones	0.322

Neither dimension shows a statistically significant bias. This is genuine new evidence targeting Assumption 1’s specific weak point, not a restatement of prior randomness tests, and it is bounded honestly: $k \leq 9$ does not establish high algebraic degree at the scale that would actually matter (the full lattice state), which is computationally infeasible to explore exhaustively here, and it is not a substitute for third-party cryptanalysis. It moves Assumption 1 from “unexamined” to “examined with one targeted technique, and not broken” — real evidence, not proof.

9.2 A Differential Cryptanalysis Attempt

The cube tester targets low algebraic degree; differential cryptanalysis (Biham and Shamir, 1990/1991) asks a different question — does some specific chosen input difference propagate with anomalously high probability. A full, formal, multi-round trail search over an 8-round Feistel network on a 262,144-bit state is not attempted here, and that boundary is stated up front rather than left for a reader to discover: that is a genuinely hard combinatorial search real cipher analysis usually automates with dedicated tools. What is tractable and exact: theta is GF(2)-linear, so differences propagate through it deterministically, and iota XORs a public constant into both sides of any difference pair, canceling exactly. Chi is the only nonlinear step, acting as a fixed 6-input local function at every cell — its exact differential distribution table, unlike the full system, is fully enumerable over all 64 local input differences.

The single highest-bias entry is the isolated- x -only difference (all five neighbor inputs unchanged), giving $P(\Delta y=1) = 1$ exactly. This is not novel: chi’s bare linear term in x forces $\Delta y = \Delta x$ whenever x alone differs, a structural property of any function of this shape (the same holds for Keccak-f’s own chi under the analogous case), and it is not exploitable in isolation. Excluding that one forced case, the remaining 62 nonzero differentials show a mean bias of 0.057 from ideal, with 48 of 62 exactly unbiased and a maximum bias of 0.25.

Whether the isolated- x case survives into a useful multi-round trail was tested directly against the real construction: starting from single-bit differences and tracking Hamming weight across 8 real Feistel rounds. Across 15 independent trials, weight reached the ideal $\sim 50\%$ by round 4–5 every time and stayed there through round 8, with no trial showing anomalously slow diffusion — theta’s diffusion does not let the one deterministic local case survive even one full round intact.

9.3 Path Validation Soundness

The path validation mechanism of Section 8 reduces cleanly to Assumption 2 (HMAC-SHA3-256’s PRF security), already established above, since it protects an optional layer above the Data Plane rather than the Data Plane itself. Under Assumption 2, for any adversary without K_{auth} , the probability of producing a value matching $\text{HMAC-SHA3-256}(K_{\text{auth}}, \text{“CTP-PATH-CHALLENGE”} \parallel R)$ for a fresh, uniformly random R is at most $\text{Adv}_{\text{HMAC}}^{\text{PRF}} + 2^{-256}$: a distinguisher against HMAC is built directly from any adversary that beats this bound, since against a truly random function the best any adversary can do is guess the 256-bit output outright. This bound does not cover the challenge rate-limiting used operationally, which is a resource-management control rather than a cryptographic property.

10 GPU ACCELERATION

The two computationally dominant operations in the Data Plane — the lattice round function and the Keccak-p permutation underlying K12 — have GPU compute-kernel implementations (via Taichi). Both were validated for correctness before being trusted, and both have since been

measured on real GPU hardware (an NVIDIA GeForce RTX 3070, CUDA 13.1, with an AMD Ryzen 9 5900X 12-core/24-thread CPU as the sequential baseline). The two results point in different directions initially, and the current design follows the evidence — described in full below, including the corrections this evidence required — rather than a uniform GPU-everything approach or an uncorrected first measurement.

10.1 Lattice Round Function

The $\theta/\chi/\iota$ round function (Section 4) is implemented as parallel compute kernels operating across the full index space of each lattice half, batched across an arbitrary number of concurrent sessions. Every cell’s update, for every session, is independent within a round, so this maps directly onto GPU thread parallelism with no synchronization required mid-round. The kernel is checked bit-for-bit against the CPU (numpy) reference across many independent random seeds and generations, and the batched multi-session form is checked bit-exact against independently-computed per-session results.

Measured on real CUDA hardware, batched execution time is effectively flat as concurrent session count increases, while sequential (CPU/numpy) execution scales linearly (Table 2). At 128 concurrent sessions, the batched GPU path is measured at approximately 88× the throughput of sequential CPU execution.

TABLE 2
Lattice round function: measured GPU vs. sequential CPU

Sessions (N)	Batched (GPU)	Sequential (CPU)	Speedup
1	6.9 ms	6.4 ms	0.9×
8	6.2 ms	51.1 ms	8.2×
32	5.6 ms	205.6 ms	37.1×
128	9.3 ms	814.8 ms	87.6×

The $N = 1$ result is informative on its own: batched and sequential are roughly at parity, which isolates the fixed cost of a single kernel launch (on the order of a few milliseconds) as the only overhead the GPU path pays before parallelism has anything to amortize it against. Every additional concurrent session beyond that point is close to free until the GPU’s own parallel capacity is exhausted, which this hardware did not reach within the tested range.

10.2 Keccak-p Permutation and Keystream Extraction

A single Keccak-p permutation’s 12 rounds are inherently sequential — each round depends on the output of the previous one, so one instance cannot itself be meaningfully parallelized. The batchable unit is instead many independent permutation instances at once, which is what K12’s chunk-based tree structure (Section 5) produces for large inputs. The kernel and the full K12 tree construction (chunking, chaining values, Sakura final-node encoding) were validated in stages against independent reference implementations — known test vectors, Python’s own `hashlib`, an independent TurboSHAKE128 implementation, and an independent KangarooTwelve implementation — at every relevant boundary

condition, including CTP’s actual operating input size, before any GPU measurement was taken.

Reaching a working, competitive GPU implementation of this component took six rounds of measurement, each surfacing a defect the previous round could not see. This process is reported in full because the methodology — treating a suspicious result as something to investigate rather than accept — is as much a finding here as the final number.

Round 1: incomplete batching. An initial version batched only the chunk-level chaining-value computation, leaving the final-node combination step (the one that actually produces the output) on a slow, unaccelerated reference implementation. Measured: 230–1500× slower than an optimized CPU baseline, worsening at smaller batch sizes.

Round 2: a kernel recompilation trap. Fixing round 1 exposed a second defect: passing a newly-allocated GPU buffer of an already-seen shape to a compute kernel triggered full recompilation rather than reuse, at roughly three orders of magnitude in wall-clock cost for a single call (confirmed directly: reusing the same buffer object measured ~ 0.1 ms/call; a fresh buffer of identical shape measured ~ 800 ms/call). Fixed by allocating GPU-resident buffers once per shape and reusing them.

Round 3: the final node was still unfused. With rounds 1–2 fixed, profiling (not assumption) showed a flat performance ratio that looked like it might reflect GPU hardware saturation. It did not: the final-node combination step — which unlike the chunk-level computation also needs a variable-length squeeze phase — was still routed through the slow reference path. Extending the fused kernel to cover absorb-and-squeeze for the final node as well, batched across sessions the same way, closed this gap.

Round 4: profiling the complete, correct pipeline. With every cryptographic step now GPU-resident and correctness re-validated, a phase-by-phase timing breakdown of the complete pipeline showed the GPU kernels themselves at under 1% of total runtime — effectively free. The remaining $\sim 99\%$ was unaccelerated Python and numpy orchestration: per-session chunking, padding, and reassembly, each looped in Python rather than vectorized across the batch.

Round 5: vectorizing the orchestration. Because K12’s padding depends only on a chunk’s length, not its content, and CTP sessions in a burst share a fixed packet size, chunking and padding were rewritten as single batch numpy operations across all sessions at once, exploiting this length-invariance instead of recomputing identical padding per session in a loop.

Round 6: a self-inflicted regression, caught the same way. The first version of the round-5 fix reintroduced a bottleneck already fixed once before (concatenating byte buffers via slow per-session joins) and added several unnecessarily large array copies. Profiling the new code before trusting it — the same discipline applied to every other component in this project — caught this before it was reported as a result.

The final, fully corrected and re-validated result reverses the earlier conclusion (Table 3): GPU-batched keystream extraction is faster than the optimized CPU baseline at every batch size tested from 512 sessions upward, and the margin is still growing at the largest scale tested rather than plateauing.

TABLE 3
Keystream extraction, final result: GPU vs. optimized CPU

Sessions (N)	Fused (GPU)	CPU (optimized)	Speedup
512	25.1 ms	33.4 ms	1.33×
1024	38.3 ms	68.1 ms	1.78×
2048	76.7 ms	142.2 ms	1.85×
4096	147.9 ms	282.1 ms	1.91×
8192	287.2 ms	569.5 ms	1.98×

10.3 Resulting Design: Both Components on GPU

Based on measurement rather than assumption, the current design places both the lattice round function and keystream extraction on GPU. An earlier draft of this document concluded the opposite for keystream extraction, based on an implementation that was, at the time, incompletely fused (Section 10.2); that conclusion is superseded by the results above, not merely updated, and the intervening reasoning is kept in this document rather than deleted, since the debugging process is itself informative about how to measure this kind of system honestly.

This single-workload-slower, batched-workload-faster pattern is not new to CTP. A US patent on GPU task consolidation (No. 8,643,656) reports the identical qualitative shape for a different encryption workload: a single job underperforms on GPU by roughly 18%, while consolidating multiple jobs reverses this to a 68% GPU improvement. Wang and Chu’s 2019 work on batch-mode Keccak on GPU independently reports the same shape for plain SHA-3. A substantial separate literature accelerates standardized block ciphers on GPU directly (Manavski 2007; Iwai et al. 2010, 2012; Li et al. 2012; Tezcan 2021), and at least two prior studies (Fei et al. 2020; Yang and Jeong 2025) investigate hybrid CPU/GPU AES workflows explicitly analogous to this document’s own hybrid-then-corrected architectural question. What is not present in that prior work, as far as this document’s authors are aware, is the specific multi-round profiling discipline reported in Section 10.2: an incompletely-fused first attempt at exactly this kind of batching measured over three orders of magnitude slower before the actual bottleneck was correctly diagnosed as unaccelerated host-side code rather than a limit of the batching principle itself.

10.4 Absolute Throughput in Context

The speedup in Table 3 is a valid, measured, internal comparison — GPU-batched execution against an optimized sequential CPU baseline processing the same workload — and should not be read as more than that. Converting the best measured result (8192 sessions, 287.2 ms, 4096 bytes of usable keystream per session) to an aggregate data rate gives approximately 117 MB/s, or under 1 Gbps. A single CPU core running contemporary hardware-accelerated AES-256-GCM or ChaCha20-Poly1305 (with AES-NI or equivalent ARM cryptography extensions) typically achieves 1–6 GB/s in published benchmarks; dedicated NIC or ASIC offload reaches tens to hundreds of Gbps. CTP’s validated GPU result is therefore roughly one to two orders of magnitude below a single core of mature, standardized software encryption, and several further orders of magnitude below hardware-offloaded deployments. The result in Section 10.2

answers a narrower, architectural question — does batching this workload across sessions on a GPU beat naive sequential CPU execution of the same workload — and the answer is yes, measured directly. It does not answer, and this document does not claim it answers, whether CTP is competitive with production cryptography in absolute terms; it is not, by a wide margin, and stating that plainly is more useful than leaving it to be inferred.

10.5 Backend Selection

GPU backend initialization was found, during development, not to fail gracefully on a system without a working backend: a naive backend request can terminate the process outright rather than raising a catchable exception, and some backend initialization paths silently substitute a different backend (e.g., falling back to CPU) without signaling that the requested one was unavailable. The current implementation probes each candidate backend inside a disposable subprocess and checks that the *resolved* backend matches the one requested before committing the main process to it, so a failed or substituted backend is detected safely rather than assumed away.

11 PROPOSED BENEFITS OF GPU-RESIDENT PROCESSING

Both the lattice and keystream-extraction results in Section 10 are now measured; the benefits below extend those measurements to claims about a full deployment, which have not themselves been measured, and that distinction is maintained deliberately throughout this section. Section 10.4’s absolute-throughput context applies to every claim in this section as well: these are claims about relative behavior (GPU vs. CPU, scaling with concurrency), not claims that CTP is competitive with production ciphers in absolute terms.

11.1 Throughput That Scales With Concurrent Load

Table 2 shows this directly for the lattice round function: batched GPU cost stays within a narrow band as concurrent sessions scale from 1 to 128, while CPU cost scales linearly. Table 3 shows a related but distinct pattern for keystream extraction: GPU throughput does not merely stay flat, it continues *improving relative to CPU* as concurrent sessions scale from 512 to 8192, with no sign of plateauing at the largest size tested. A deployment’s aggregate cost under concurrent load would be expected to follow one of these two favorable shapes for both components now, up to the point where a given GPU’s parallel capacity is saturated — a point neither test reached.

11.2 Headroom for a Larger Security Margin

Because both components’ dominant costs are measurably cheap to run in parallel, there is room to increase the design’s internal security parameters — a larger lattice, more Feistel rounds, a higher-degree nonlinear step, a larger extraction sponge — without the proportional slowdown those changes would cause in a CPU-bound implementation. This is the sense in which GPU offload could enable “bigger” encryption: not a larger key, but a larger and more thoroughly-mixed

internal state, affordable specifically because the components that scale with it are now both shown to run efficiently in parallel.

11.3 Reduced Host-Memory Exposure

Keeping state resident in GPU memory across an operation, rather than repeatedly copying it to host RAM, narrows the window in which secret-derived material exists in host-visible memory. This is a proposed reduction in exposure, not a proven confidentiality boundary: GPU memory isolation guarantees vary by platform and driver, and this remains an open question (Section 12) rather than a settled property of the design. With both the lattice and keystream extraction now GPU-resident (Section 10.3), this benefit potentially applies across more of the encryption pipeline than earlier drafts of this design supported, though the underlying platform-security question is unchanged and still unresolved.

11.4 Natural Fit for High-Concurrency Deployments

A server terminating many simultaneous encrypted connections presents a workload shape matching what both Table 2 and Table 3 measured, for both components of the pipeline now rather than one. This is the deployment scenario in which GPU offload has a demonstrated case, as distinct from a single low-traffic session, where the pipeline’s sequential dependencies dominate and GPU offload offers little regardless of component.

12 KNOWN LIMITATIONS

- **GPU measurement so far covers one hardware configuration.** The results in Section 10 are from a single NVIDIA GeForce RTX 3070 (CUDA 13.1); relative performance, particularly the point at which either component’s GPU throughput would saturate, has not been characterized across other GPUs, driver versions, or Taichi backends (Vulkan, Metal). Keystream extraction’s speedup was still growing at the largest session count tested (8192); whether it continues, plateaus, or reverses beyond that point is not established.
- **CTP’s absolute throughput remains far below contemporary encryption.** Section 10.4 states this directly: the validated GPU speedup is an internal comparison, not evidence of competitiveness with production ciphers. CTP’s best measured aggregate throughput sits one to two orders of magnitude below a single CPU core of hardware-accelerated AES-256-GCM or ChaCha20-Poly1305, before considering hardware offload.
- **The formal argument in Section 9 is conditional, not unconditional.** It proves CTP’s Data Plane secure *given* that the keystream generator behaves as a PRF and HMAC-SHA3-256 is SUF-CMA. The second is well-supported; the first is specific to this project’s own novel construction and has not been reviewed by anyone else. The cube-tester and differential-analysis results are evidence toward that assumption from two distinct attack families, not proof of it, and

neither covers a full-scale, automated, multi-round trail search.

- **No third-party cryptanalysis has been performed.** A from-scratch implementation of the core NIST SP 800-22 statistical tests, the cube-tester attempt in Section 9, and the other structural validation throughout this document are evidence gathered by the authors, not independent review.
- **GPU hardware security posture is unresolved.** Memory isolation guarantees, buffer-zeroing behavior between kernel launches, and the applicable adversary model all vary by platform and have not been established here.
- **The connection ID and path validation mechanism has been tested in a synchronous simulation, not a concurrent network implementation.** Real deployment requires handling multiple outstanding validation attempts and asynchronous challenge delivery correctly.
- **This is a research prototype.** It should not be used to protect real data.

13 CONCLUSION

CTP’s diffusion layer is built specifically to be a good match for parallel hardware, and that claim now has a measurement behind it rather than only an architectural argument: on real GPU hardware, batched lattice diffusion across 128 concurrent sessions ran at roughly $88\times$ the throughput of sequential CPU execution, with batched cost staying nearly flat as session count grew from 1 to 128. Keystream extraction reached the same qualitative conclusion by a longer and more instructive path: six rounds of profiling-driven fixes, several of them invisible until the previous round’s fix exposed what was underneath it, turned an initial $1500\times$ slowdown into a final $1.3\text{--}2.0\times$ *speedup* over an optimized CPU baseline, with the margin still growing at the largest scale tested. The single most important methodological result in that process was not any specific bug, but the demonstration that a flat, hardware-looking performance ceiling was in fact over 99% unaccelerated host-side code, findable only by phase-level profiling and not by aggregate timing — a distinction that matters for how any GPU-cryptography result should be evaluated, not just this one. This document also gives CTP’s Data Plane a formal, reduction-based security argument: nonce-based authenticated encryption security follows from standard encrypt-then-MAC composition, conditional on the keystream generator behaving as a PRF, an assumption two targeted attacks — a cube tester probing algebraic degree, and an exact differential distribution table analysis probing high-probability differentials — each attempted, and failed, to break at bounded scope. The connection-identification layer added for multiplexing and migration followed the same discipline in miniature: a naive first version was tested against the specific attack its design predicted rather than assumed safe, the attack succeeded exactly as predicted, and the fix was re-tested against the identical attack until it failed cleanly, with a formal soundness argument reducing its guarantee to the same MAC assumption used elsewhere. Both the performance and security results are reported alongside an explicit reminder of their limits: CTP’s absolute throughput

remains one to two orders of magnitude below a single core of contemporary hardware-accelerated encryption, and its security rests on assumptions no third party has reviewed. Stating both plainly is part of this document's purpose, not an afterthought. CTP remains a research artifact for further study, not a cipher ready for production use.